

Comparison of Array and ArrayList in Java

Feature	Array	ArrayList
Definition	Fixed-size data structure to store elements of the same type.	Resizable array implementation of the List interface.
Size	Fixed size, defined at the time of creation.	Dynamic size, can grow or shrink as elements are added or removed.
Memory Allocation	Memory is allocated at the time of initialization and cannot be changed later.	Memory is allocated dynamically. When the internal array is full, it is resized automatically.
Type of Data Stored	Can store elements of any data type (primitives or objects).	Can only store objects, but for primitive types, autoboxing is used to store their wrapper classes (e.g., Integer for int, Double for double).
Accessing Elements	Accessed by using indices (e.g., <code>array[0]</code>)	Accessed by using indices (e.g., <code>arrayList.get(0)</code>)
Performance	Faster in accessing elements due to fixed size and direct memory access.	Slightly slower due to dynamic resizing and overhead of method calls.
Null Elements	Can contain <code>null</code> for reference types, but not for primitive types.	Can contain <code>null</code> values.
Storage of Primitives	Can store primitive types directly (e.g., <code>int[]</code> , <code>char[]</code>).	Cannot store primitive types directly; uses wrapper classes (e.g., <code>Integer</code> , <code>Character</code>).
Modification	Elements can be modified using indices (e.g., <code>array[0] = 5</code>).	Elements can be modified using the <code>set()</code> method (e.g., <code>arrayList.set(0, 5)</code>).
Adding/Removing	Adding or removing elements is not supported. You must reallocate a new array.	Supports adding and removing elements dynamically using <code>add()</code> and <code>remove()</code> methods.
Storage Efficiency	More memory efficient as it stores	Less memory efficient due to extra overhead for dynamic resizing and

Feature	Array	ArrayList
	only the elements in a contiguous block.	object wrapping.
Type Safety	Can be type-safe when using generics (e.g., <code>int[]</code> , <code>String[]</code>).	Type-safe when using generics (e.g., <code>ArrayList<Integer></code> , <code>ArrayList<String></code>).
Support for Collections Framework	Does not directly implement any collection interfaces.	Implements the <code>List</code> interface, which is part of the Java Collections Framework.
Resizing	Cannot resize once defined.	Resizes automatically when the number of elements exceeds its capacity.
Methods	Limited methods, mainly for accessing elements and length (<code>length</code> attribute).	Provides a rich set of methods like <code>add()</code> , <code>remove()</code> , <code>get()</code> , <code>set()</code> , <code>size()</code> , and more.

Key Differences Summary:

- **Array** is a fixed-size collection of elements, while **ArrayList** is a dynamic, resizable collection.
- Arrays can store primitive types directly, whereas **ArrayList** stores objects and requires autoboxing for primitive types.
- **Array** is more memory efficient and faster for fixed-size data, but **ArrayList** offers more flexibility for dynamic data structures, with methods for adding, removing, and resizing elements.